

Lab Report No 2
Introduction to OpenCV Library w.r.t. Image
Processing Basic Commands
Digital Image Processing



Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

[Aqsa Shah]

[21-CE-039]

Submitted to:

Engr. Hareem Khan

Dated:

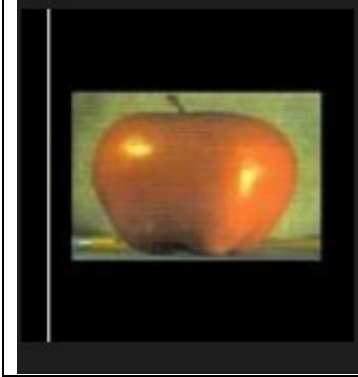
3rd Oct, 2024

Department of Computer Engineering,
HITEC University, Taxila

Lab Task No 1:

Solution:

Brief description (3-5 lines)
Now create a generic code that creates a border around any landscape image as shown below. The length of right and left borders must be 10% of the original horizontal length of the image. The length of upper and lower border must be such that the image now have same number of rows as columns. Save the image.
The code
<pre>from PIL import Image # Use a raw string to avoid issues with backslashes in the file path image_path = r"D:\BSCOMPUTER ENGINEERING\4th YEAR OF COMPUTER ENGINEERING\7th SEMESTER\DIGITAL IMAGE PROCESSING\PROGRAMS OF DIGITAL IMAGE PROCESSING\Screenshot 2024-10-09 091542.jpg" # Load the image image = Image.open(image_path) # Get the original dimensions original_width, original_height = image.size # Calculate the borders left_right_border = int(0.10 * original_width) # Calculate the required final size to make it a square (rows = columns) final_size = original_width + 2 * left_right_border # Calculate the upper and lower borders to make it a square upper_lower_border = (final_size - original_height) // 2 # Create a new image with the final size and black borders new_image = Image.new("RGB", (final_size, final_size), (0, 0, 0)) # Paste the original image in the center with borders new_image.paste(image, (left_right_border, upper_lower_border)) # Save the new image new_image.save("image_with_borders.jpg")</pre>
The results (Screenshot)



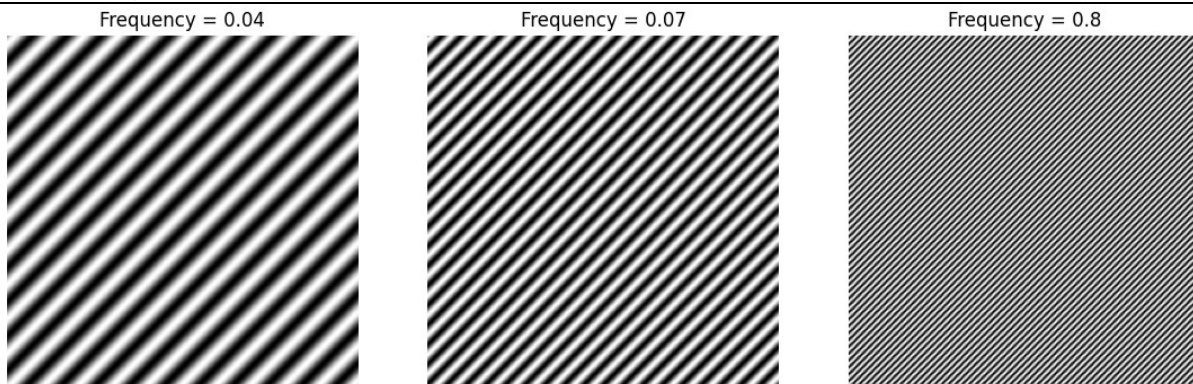
Lab Task No 2:

Solution:

Brief description (3-5 lines)
Using the following formula $f(i, j) = \sin(2\pi f(i + j))$ where i and j are indices of a pixel, draw an image with different frequencies (input from user).
<pre>import numpy as np import matplotlib.pyplot as plt # Function to generate the image with sine wave pattern def generate_sine_image(frequency, size=256): image = np.zeros((size, size)) for i in range(size): for j in range(size): image[i, j] = np.sin(2 * np.pi * frequency * (i + j)) return image # Function to display the images with different frequencies def plot_images(frequencies, size=256): plt.figure(figsize=(15, 5)) for idx, freq in enumerate(frequencies): plt.subplot(1, len(frequencies), idx + 1) image = generate_sine_image(freq, size) plt.imshow(image, cmap='gray', extent=(0, size, 0, size)) plt.title(f'Frequency = {freq}') plt.axis('off')</pre>

```
plt.show()
# Ask user for frequencies
frequencies = [float(x) for x in input("Enter frequencies separated by
spaces: ").split()]
# Generate and display the images
plot_images(frequencies)
```

The results (Screenshot)



Lab Task No 3:

Solution:

Brief description (3-5 lines)

Write Python functions that can create the images given below. Code them in such so that the size of the image itself are entered by the user.

The code

```
from PIL import Image, ImageDraw
import math
def create_image1(size):
    img = Image.new('RGB', (size, size), (255, 255, 255))
    draw = ImageDraw.Draw(img)
    draw.rectangle([(0, 0), (size-1, size-1)], outline=(0, 0, 0))
    img.save('image1.png')
def create_image2(size):
    side_length = size // 2
    img = Image.new('RGB', (size, size), (0, 0, 0))
    draw = ImageDraw.Draw(img)
```

```

draw.rectangle([(side_length//2, side_length//2), (size-
side_length//2-1, size-side_length//2-1)], fill=(255, 255, 255))
img.save('image2.png')
size = int(input("Enter the size of the image: "))
create_image1(size)
create_image2(size)

```

The results (Screenshot)



IMAGE 1.PNG

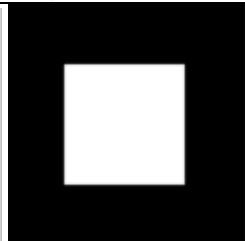


IMAGE2 .PNG

Conclusion

In conclusion, The OpenCV library provides essential image processing commands, including image loading, manipulation, and transformations. Its functionality includes image filtering, edge detection, and feature extraction, supporting diverse computer vision applications. Its accessible API and extensive documentation enable seamless integration into projects, enabling developers to implement image processing solutions efficiently.